Agent 测试时计算扩展

相关论文

1. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling
2. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
3. Archon: An Architecture Search Framework for Inference-Time Techniques

固定模型下的推理计算

训练阶段的 scaling law 描述训练计算量与模型性能的关系。

在模型参数固定以后，能否通过更多采样、搜索、修订和验证，提高同一个模型的最终成功率？

1. **Large Language Monkeys**　重复采样为什么呈现可预测的 scaling law?
2. **Compute-optimal test-time scaling**　并行采样和串行修订应如何分配预算?
3. **Archon**　如何自动搜索由 generator、critic、ranker、fuser 和 verifier 组成的推理架构?

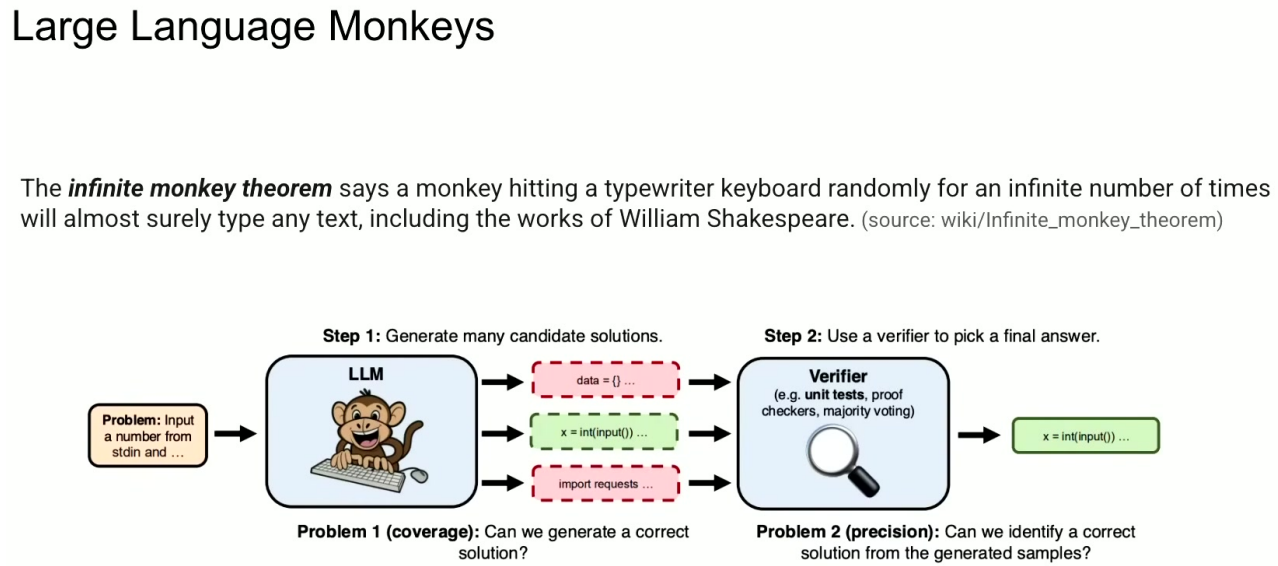
候选覆盖率与选择策略

生成决定候选集能否覆盖正确解，**验证**决定能否识别正确解，**预算分配**决定有限计算能否用在收益最高的操作上。

指标或策略	**含义**
pass@1	系统只交付一个答案时的最终准确率
pass@k / coverage	k 个候选中至少存在一个正确答案的概率
best-of-N	生成 N 个候选后，由 verifier 选择一个答案
majority@N	按答案频率投票，假设正确答案最常出现

系统最终只交付一个答案，因此需要分别测量 coverage 与 pass@1。比较策略时，还需要权衡候选质量、选择质量与计算成本。

Large Language Monkeys 的生成与验证流程



Repeated sampling 从同一模型独立采样多个候选，再交给 verifier 选择。单元测试、编译器和 proof checker 可为代码与形式化证明提供自动反馈。

单题 coverage 随采样次数饱和

对问题 i 独立采样 k 次时，oracle coverage 等于至少一次采样成功的概率。

$$C_i(k) = 1 - (1 - p_i)^k.$$

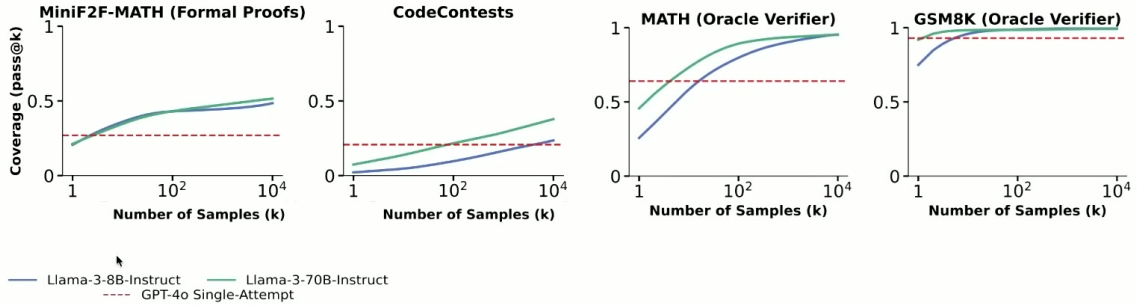
- p_i　模型对问题 i 的单次成功概率。
- k　采样数。
- $C_i(k)$　候选集合至少包含一个正确解的概率。

当 $0 < p_i < 1$ 时，初始采样的边际收益最大，之后逐步下降。

小模型为何能通过采样追上强模型

Models Improve Drastically with Just Repeated Sampling!

Llama-3-8B outperforms GPT-4o on various reasoning benchmarks if sampled more!



*Due to the large compute cost, we generated results c random subsets of datasets

9 Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

大量采样缩小了小模型与强模型单次推理的差距。实验覆盖数学、代码与形式化证明。

能力被“放大”还是被“发现”？采样越多，越容易找到已有的低概率正确轨迹。

数据集平均 coverage 的经验拟合

平均 coverage 随采样数变化，可用 exponentiated power law 拟合。

$$\bar{C}(k) \approx 1 - \exp(-ak^b),$$
$$a > 0, \quad 0 < b < 1.$$

- $\bar{C}(k)$　数据集上的平均 coverage。
- a　与模型和任务总体难度相关的尺度参数。
- b　控制 scaling 速度的指数。
- k　推理采样预算。

Scaling Laws for Inference

- The relationship between coverage (accuracy) and the number of samples can be modelled using an *exponentiated power law*
- Similar to pretraining, this allows us to predict the performance as a function of scaling compute, but this time scaling happens only at inference time

$$c = \exp(ak^b)$$

k: number of samples
a & b: curve fitting parameters
c: coverage or fraction of problems solved by at least one sample

e Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
dley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

不同规模模型的 inference scaling

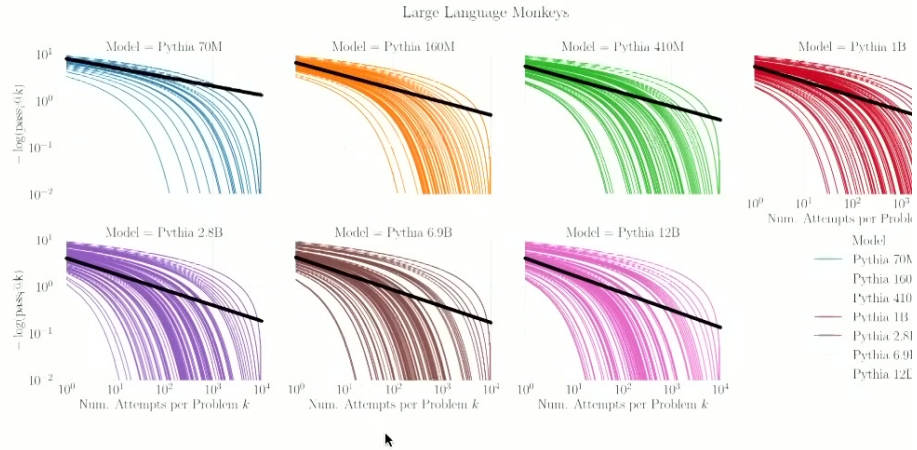
How Do Large Language Monkeys Get Their Power (Laws)?

For a SINGLE problem, we should expect -log(pass@k) to scale exponentially with k:

$$\begin{aligned}\text{pass}_\text{i}@\text{k} &= \mathop{\mathbb{E}}_{k \text{ Attempts}}\Big[1 - \mathbb{I}[\text{All } k \text{ Attempts Fail}]\Big] \\ &= 1 - \prod_{j=1}^{k} \mathop{\mathbb{E}}_{1 \text{ Attempt}}\Big[\mathbb{I}[j\text{-th Attempt Fails}]\Big].\end{aligned}$$

$$\text{pass}_\text{i}@\text{k} = 1 - (1 - \text{pass}_\text{i}@1)^k.$$

$$-\log(\text{pass}_\text{i}@\text{k}) \approx (1 - \text{pass}_\text{i}@1)^k.$$



-log(avg pass@k): Power Law (Polynomial) Scaling with k

-log(pass_i@k): Exponential Scaling with k

How can we reconcile these?

w Do Large Language Monkeys Get Their Power (Laws)?,
lan Schaeffer, Joshua Kazdan, John Hughes, Jordan Juravsky, Sara Price, Aengus Lynch, Erik Jones, Robert Kirk, Azalia Mirhoseini, Sanmi Koyejo,
ML 2025

为什么对问题取平均后会出现幂律？

单题 coverage 按 $1-(1-p_i)^k$ 指数饱和。许多模型偶尔才能解出的难题在 p_i 接近零的位置形成长尾。

若单次成功概率在 $p \to 0$ 附近具有密度

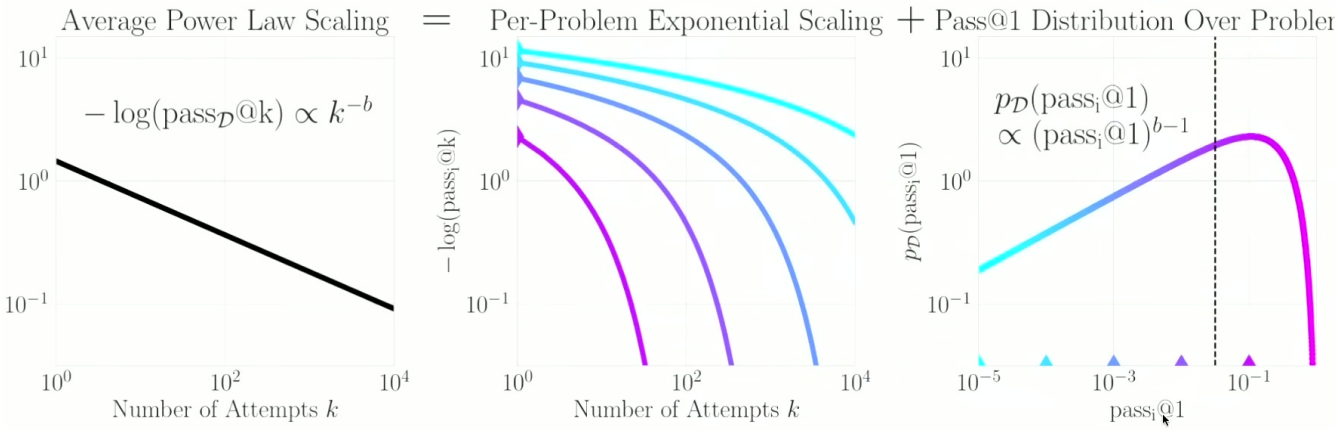
$$f(p) \propto p^{\alpha-1},$$

其中 α 是靠近零点的长尾指数，则数据集平均失败率近似为

$$1-\bar{C}(k) = \mathbb{E}_p[(1-p)^k] \approx \int_0^\infty e^{-kp} p^{\alpha-1}\, dp \propto k^{-\alpha}.$$

长尾决定采样收益何时饱和

Theorems: Long Tail of Hard Problems is Sufficient and Necessary to Create Power Law Scaling in Avg Pass@k



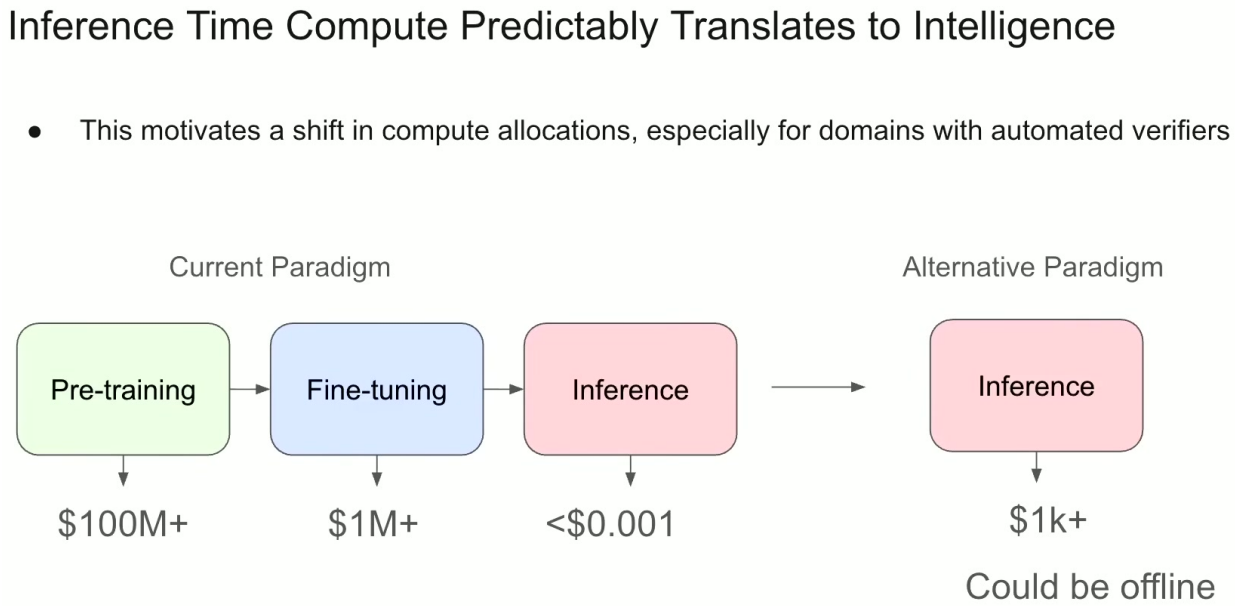
ıtuition: For -log(avg pass@k) to fall polynomially with the number of attempts k, if each problem's ɔg(pass_i@k) falls exponentially with k, then a long tail of extremely hard problems is necessary to "drag ut" the exponential curves

w Do Large Language Monkeys Get Their Power (Laws)?,

lan Schaeffer, Joshua Kazdan, John Hughes, Jordan Juravsky, Sara Price, Aengus Lynch, Erik Jones, Robert Kirk, Azalia Mirhoseini, Sanmi Koyejo, ML, 2025

如果题目的 p_i 普遍较高，少量采样后就会饱和。如果大量题目的 p_i 极低但非零，收益会延续到很大的 k。推理 scaling 也取决于 benchmark 的难度分布。

预训练与推理的预算权衡



如果基础模型偶尔能生成正确解，且 verifier 能识别这类候选，按请求增加推理计算就可能提高成功率。对少量高价值请求，这可能比训练和部署更大的模型更经济。

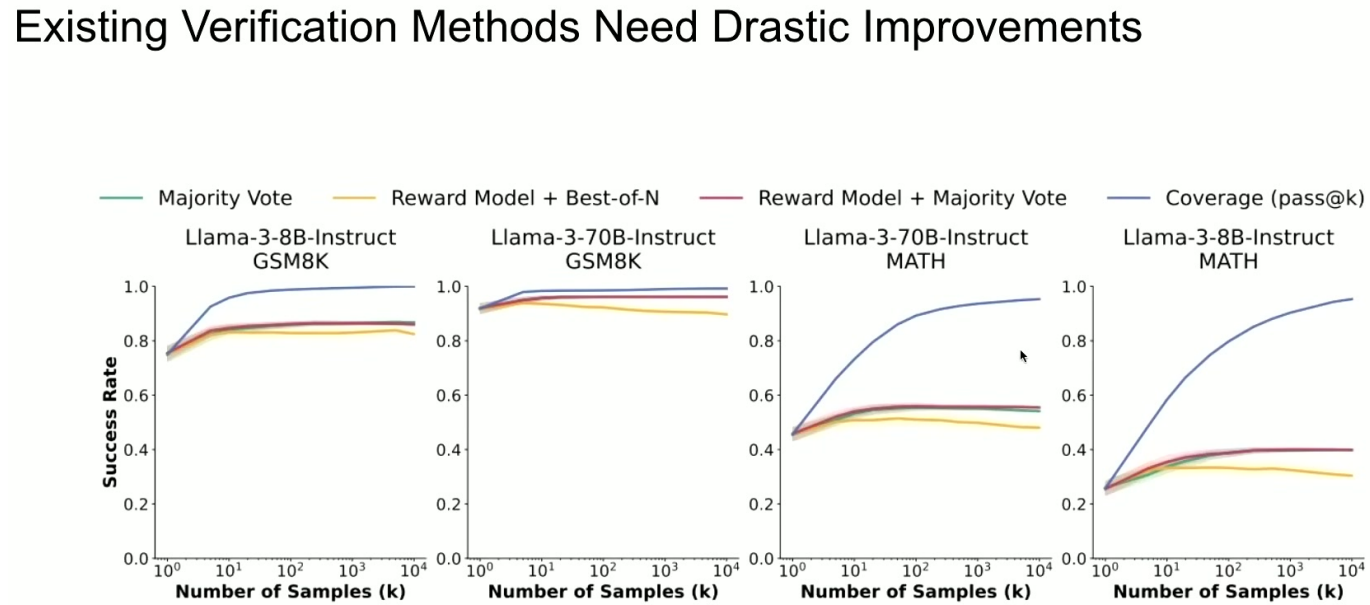
Generation-Verification Gap

Oracle coverage 是固定候选集下的准确率上界。只要候选集中有正确答案，知道 ground truth 的 oracle verifier 就能选中它。

$$G(k) = C_{\text{oracle}}(k) - A_{\text{selector}}(k).$$

- $C_{\text{oracle}}(k)$　候选集合中存在正确答案的比例。
- $A_{\text{selector}}(k)$　真实选择器交付正确答案的比例。
- $G(k)$　generation-verification gap。

候选覆盖率与实际准确率的差距



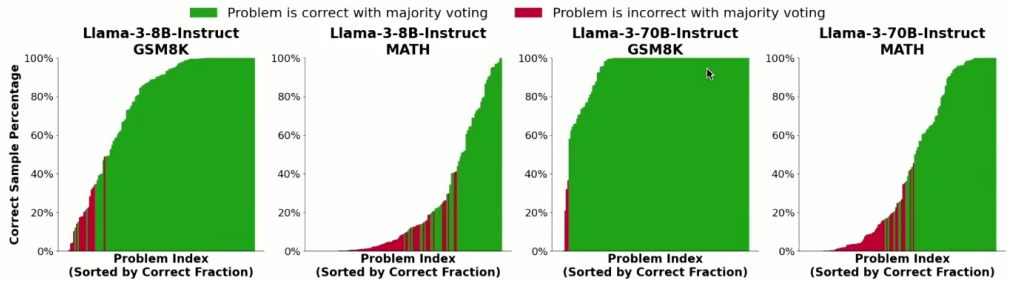
e Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
dley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

实际选择器通常使用多数投票、reward model 或 LLM judge，最终准确率可能远低于 oracle coverage。

为什么 majority vote 会失败

The Correct Generations Could Be Very Rare

So, it makes sense that majority voting doesn't work well across the board!



e Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
dley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

Self-consistency 依赖正确答案比任一错误答案更常出现。如果模型反复生成同一种错误，增加采样会让投票结果更稳定地偏向该错误。候选间的一致性反映答案出现的频率，正确性仍需验证。

验证器应具备什么性质

- **Soundness**　拒绝明显错误的候选。
- **Recall**　尽可能识别候选中的稀有正确解。
- **Calibration**　评分应反映实际成功概率。
- **Robustness**　对候选的风格、长度和表面自信保持稳健。
- **Cost**　验证成本应显著低于重新生成或人工检查。

形式化证明和代码执行可借助符号系统或真实环境。开放式任务更难低成本获取 ground truth，通常依赖 learned reward model 或延迟反馈。

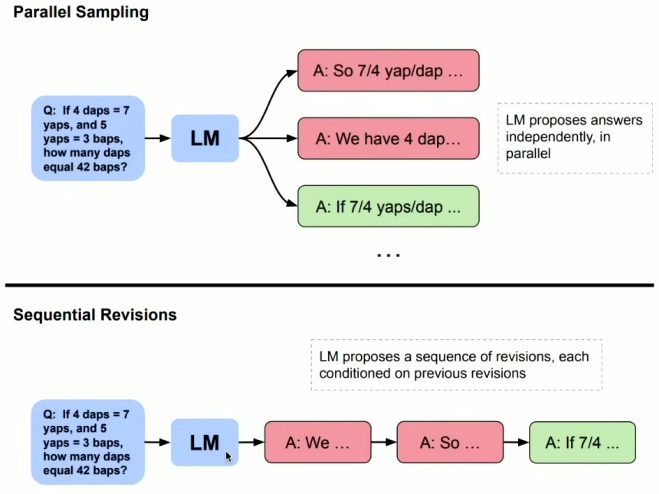
生成器变强或采样增多时，候选分布可能更复杂，稀有正确解也可能更难识别。

Compute-Optimal Test-Time Scaling

并行采样（Parallel sampling）
从同一 prompt 独立生成多个候选，扩大搜索宽度。错误不会跨候选传播。

串行修订（Sequential revision）
读取中间结果并逐步修订，增加单条轨迹的搜索深度。早期错误可能影响后续修订。

Parallel Sampling vs Sequential Revisions



Outcome Reward Model 与 Process Reward Model

ORM 只评价最终答案，适合 best-of-N。信用分配较粗。

PRM 逐步评价推理过程，为 beam search 和提前剪枝提供信号。

对包含步骤 $z_{1:T}$ 的轨迹，PRM 给出局部得分

$$r_t = r(z_t \mid z_{<t}, x),$$

帮助搜索器在生成完整答案前保留高潜力分支。

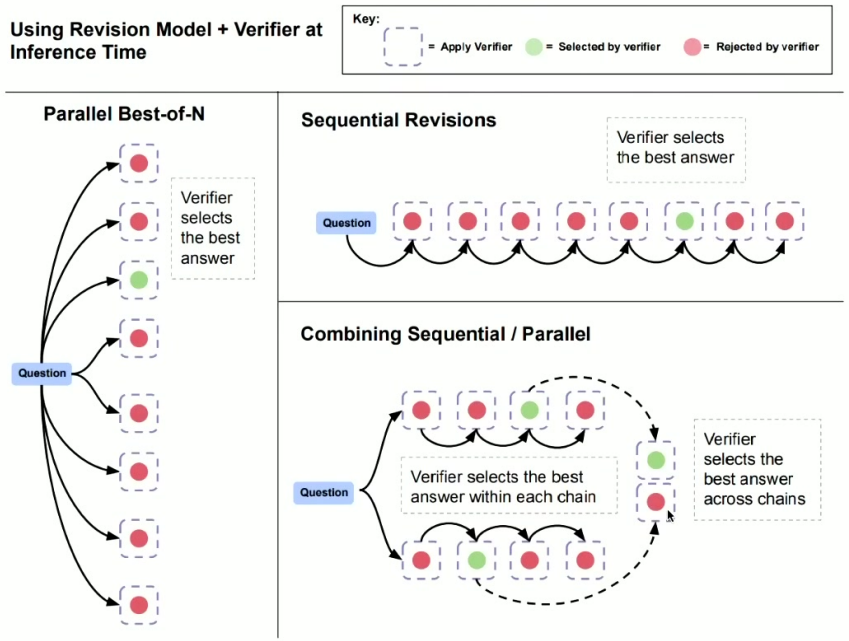
Type of Reward Models

When we evaluate the model's responses during test time:

- Outcome Reward Model assigns a score to the final answer when it is correct.
- Process Reward Model assigns a score for each step in the model's response and not just the final answer.

组合搜索宽度与深度

Compute Optimal Strategy:
Combine Parallel Sampling + Sequential Revisions



系统可以先并行探索多个初始候选，再对高分候选进行多轮串行修订。

单题总推理预算

并行采样、串行修订与验证共用预算 B。

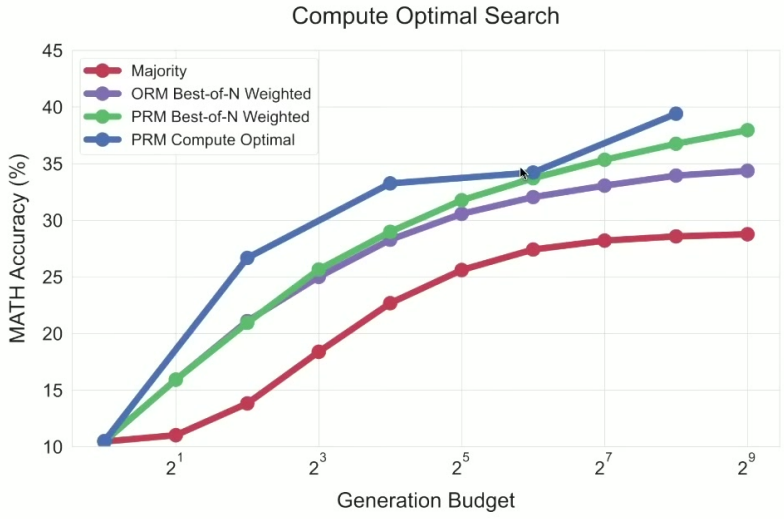
$$B = n_{\text{parallel}} \cdot c_{\text{sample}} + n_{\text{revise}} \cdot c_{\text{revise}} + c_{\text{verify}}.$$

- n_{parallel}　并行候选数。
- n_{revise}　串行修订轮数。
- $c_{\text{sample}}, c_{\text{revise}}, c_{\text{verify}}$　各操作成本。

PRM 指导下的搜索收益

Experimental Results: Test-time Search Against a PRM Verifier

Significant early improvements over best-of-N from compute-optimal scaling

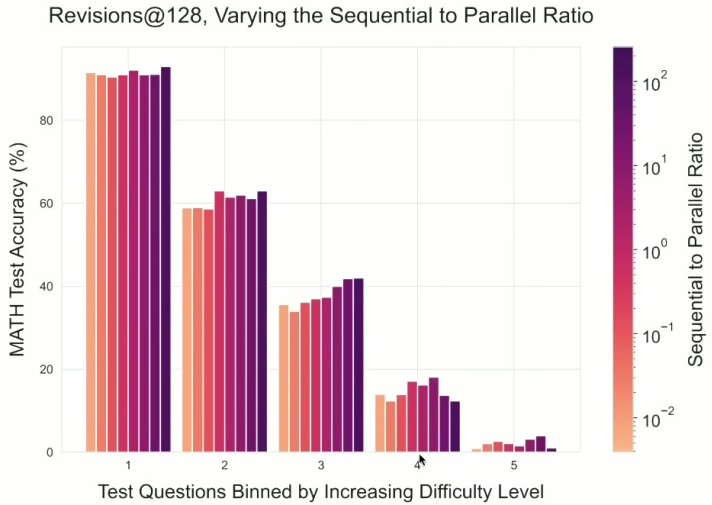


在 PRM 指导下，compute-optimal search 的早期收益高于纯 best-of-N。

最优预算分配随问题难度变化

Experimental Results: Varying the Sequential/Parallel Ratio

Using verifier-based selection, we see that the easier questions attain the best performance with full sequential compute. On the harder questions, there is an ideal ratio of sequential to parallel test-time compute.

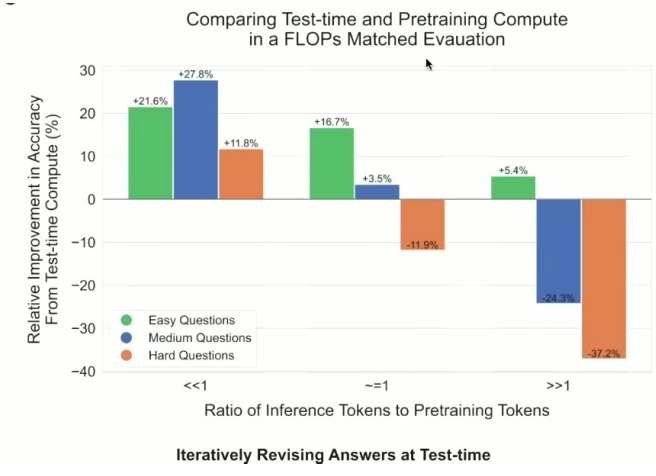


简单题往往已有接近正确的初始解，串行修订可以快速纠正。困难题的初始解可能落入错误思路，需要并行探索不同方向。

测试时 FLOPs 何时胜过预训练 FLOPs

Experimental Results: Test-time vs Pretraining FLOPs

- Easy, medium questions: additional test-time compute often preferable to scaling pretraining.
- Most challenging questions: very little benefits from scaling up test-time compute.



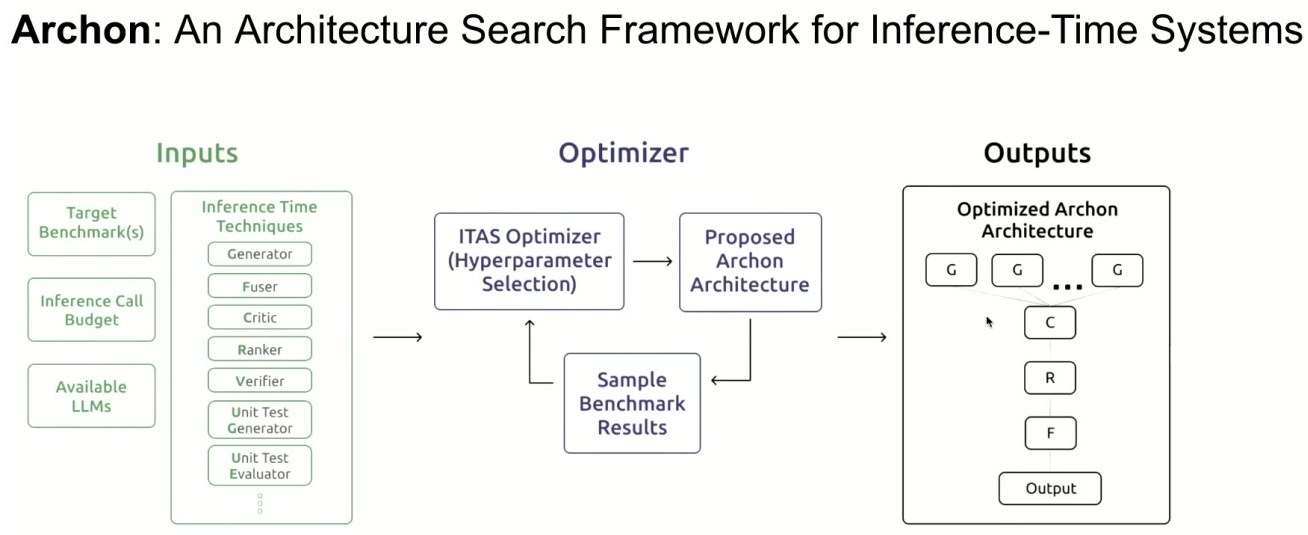
中等难度问题常从 test-time compute 获益。基础模型几乎采样不到正确轨迹时，增加搜索的收益有限，扩大预训练或改进训练数据更可能改善表现。

在线预算分配

- 过度串行会把预算耗在修补方向错误的轨迹上。
- 过度并行会产生大量浅层候选，反馈利用不足。

在线调度根据问题难度、模型能力和 verifier 质量估计边际收益，据此选择继续采样、修订、验证或停止。

Archon 将推理系统视为可搜索架构



chon: An Architecture Search Framework for Inference-Time Techniques, Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum aru, Hristo Todorov, Etash Guha, E Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, Azalia Mirhoseini

给定目标任务、候选模型、可用操作和成本约束，Archon 借用神经网络的分层结构组织推理操作，搜索各层的操作组合、顺序、模型来源和调用次数，得到一张执行图。

Archon 操作的输入、输出与成本

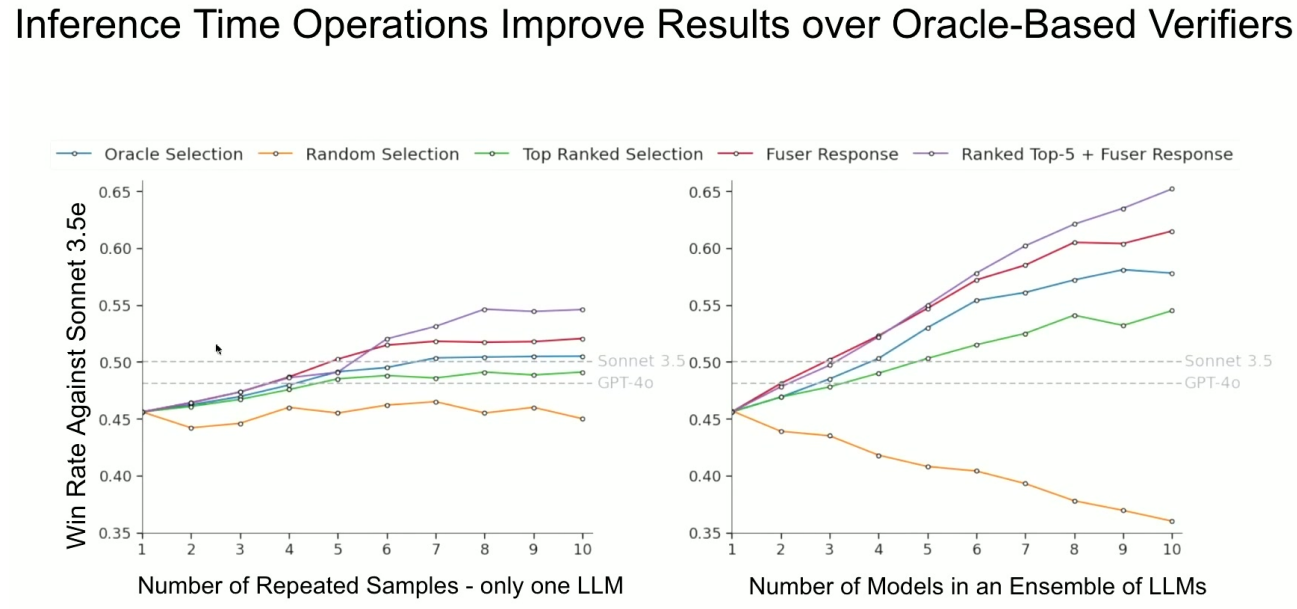
Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

可组合操作及主要风险

操作	功能	主要风险
Generator	产生一个或多个候选	缺乏多样性、重复错误
Critic	指出候选缺陷与遗漏	批评不准确或只关注表面形式
Ranker	对候选排序	位置偏差、长度偏差、judge 偏差
Fuser	综合多个候选的互补部分	把错误也融合进去
Verifier / Tests	检查约束与正确性	测试覆盖不足、奖励投机

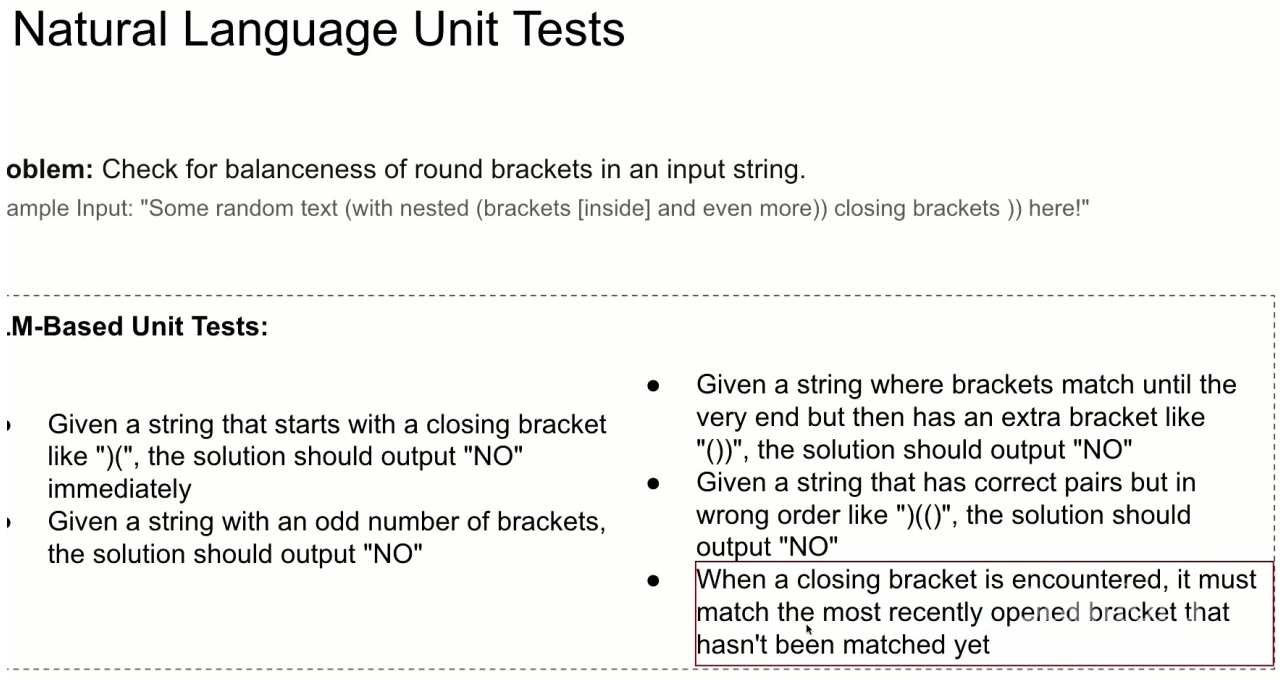
Fusion 为什么可能超过 oracle selection



chon: An Architecture Search Framework for Inference-Time Techniques, Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum aru, Hristo Todorov, Etash Guha, E Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, Azalia Mirhoseini

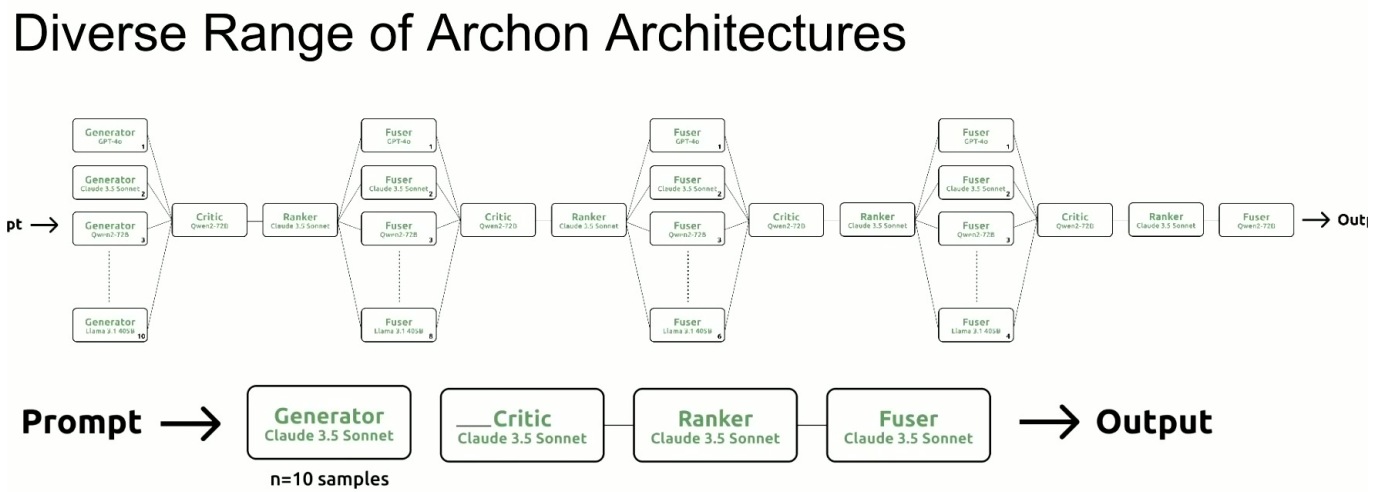
Oracle selection 的上界适用于固定候选集。Fuser 可以通过额外的模型调用，把多个候选中的互补信息融合为新答案，从而改变候选空间。

自然语言单元测试



缺少可执行的 ground truth 时，Archon 让模型生成自然语言测试来检查候选。模型可能遗漏关键需求，因此可以结合用户要求和领域规则构建检查条件。

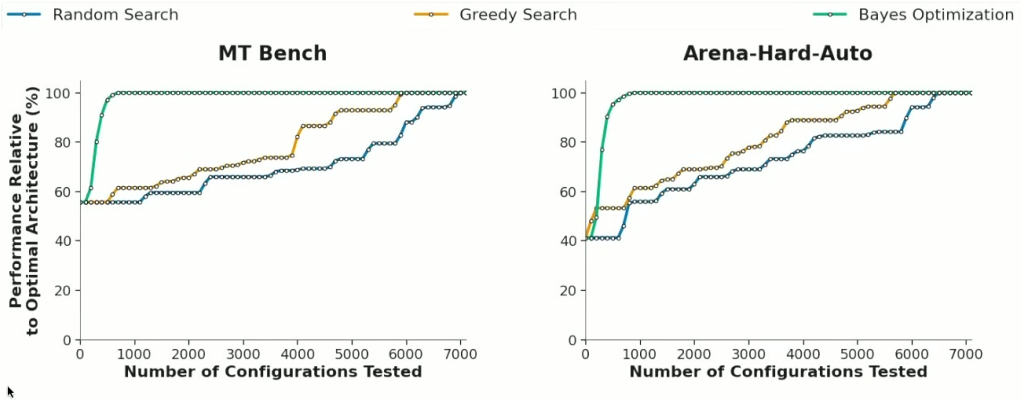
多层推理架构



多个 generator 可以并行采样，由 critic 识别缺陷，ranker 选择候选，fuser 综合答案，再进行下一轮批评与融合。

多层架构的贝叶斯搜索

Inference Time Architecture Search with Bayesian Optimization



Objective: Find the optimal configuration of LLMs for target benchmark(s), given:

- Available LLM(s)
- Inference Time Ops
- Inference Call Budget

层数、操作类型、模型和超参数共同扩大搜索空间。实验中，贝叶斯优化比随机或贪心搜索更高效地找到高性能配置。

架构搜索的目标函数

配置 a 的综合效用同时衡量质量、成本与延迟。

$$J(a) = \mathrm{Quality}(a) - \lambda_c \mathrm{Cost}(a) - \lambda_l \mathrm{Latency}(a).$$

- $\mathrm{Quality}(a)$　目标 benchmark 上的准确率或偏好分。
- $\mathrm{Cost}(a)$　模型调用和 token 成本。
- $\mathrm{Latency}(a)$　关键路径延迟。
- λ_c, λ_l　部署场景对成本与延迟的权重。

Archon 在不同模型池上的实验结果

Archon's Strong Performance with Open Source, Closed Source, and All Source Models

Archon outperforms GPT-4o and Claude-3.5-Sonnet in **pass@1** accuracy/win-rate by an average of 14.1% on instruction-following, reasoning, math, and coding tasks.

			MT Bench	Alpaca Eval 2.0	Arena Hard Auto	MixEval Hard	MixEval	MATH
	LLM Systems / ARCHON	Infer. Calls	W.R.	L.C. W.R.	W.R	Acc.	Acc.	Pass @1
	GPT-4o	1	44.2%	57.8%	80.6%	63.4%	87.5%	73.2%
	Claude 3.5 Sonnet	1	N/A	52.7%	81.4%	68.7%	89.1%	73.1%
	Llama 3.1 405B	1	44.1%	40.7%	64.5%	66.0%	88.2%	75.2%
	MoA	19	51.6%	65.4%	84.5%	62.3%	86.9%	73.9%
	MoA Lite	7	45.6%	59.6%	88.3%	60.9%	86.4%	71.8%
Archon – Open Src.	General Purpose	35	67.2%	63.3%	85.6%	65.3%	86.2%	76.6%
Archon – Open Src.	Task Specific	44	71.1%	67.1%	89.6%	67.5%	88.8%	81.9%
Archon – Closed Src.	General Purpose	32	72.7%	63.9%	86.2%	67.5%	87.2%	77.9%
Archon – Closed Src.	Task Specific	40	77.0%	68.9%	90.5%	72.6	89.5%	81.6%
Archon – All Src.	General Purpose	35	76.2%	66.4%	89.8%	69.8%	87.3%	79.3%
Archon – All Src.	Task Specific	39	**79.5%**	**69.0%**	**92.5%**	**72.7%**	**89.7%**	**82.1%**

Grad: Automatic "Differentiation" via Text Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, es Zou

ture-of-Agents (MoA) Enhances Large Language Model Capabilities Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, James Zou, 2024

Archon 在开源、闭源与混合模型池上取得稳定提升。

架构搜索的收益与代价

Archon 把 prompt engineering 扩展为系统架构搜索，在成本约束下优化多个模型调用之间的数据流和控制流。

- 搜索结果可能过拟合 benchmark。
- 更复杂的架构可能增加延迟和维护成本，降低系统的可解释性。

哪些任务适合增加推理预算？

1. 基础模型是否偶尔能产生正确答案，即 $p > 0$?
2. 是否有成本低于生成、且能可靠筛选候选的 verifier?
3. 请求价值是否足以覆盖额外 token、延迟和能耗?
4. 候选能否并行生成，还是必须串行读取环境状态?
5. 失败动作是否可逆，能否安全重试?

何时停止推理

- Verifier 分数超过可信阈值。
- 多个独立 verifier 达成一致。
- 新候选的边际质量提升低于成本。
- 连续修订没有改变关键错误。
- 达到延迟、费用或安全上限。

在线预算控制器

固定 best-of-64 为每个请求生成 64 个候选。在线控制器先用少量计算估计问题难度和候选分布，再按预期收益分配预算。

- 简单请求可以直接返回。
- 困难但可验证的请求可以扩大搜索。
- 当前模型几乎无法求解时，可以切换到更强模型或转人工。

可观测性与端到端评估

记录每个候选的生成模型、随机种子、token 数、验证分数、选择理由、工具结果和停止原因，有助于区分生成失败、验证失败、搜索策略失败和预算不足。

测试时扩展系统需要明确候选空间、验证信号、预算函数和停止规则。端到端评估同时衡量 **pass@1**、**推理成本与延迟**。

端到端收益

单次成功率呈长尾分布时，重复采样带来可预测的覆盖率增益。实际准确率取决于验证器从候选中选中正确解的能力。Compute-optimal scaling 按难度调节并行探索和串行修订的比例，Archon 则搜索多种推理操作的组合。

交付质量可用下式作直观近似。

$$\begin{aligned}\text{Delivered Quality} = \text{Generation Coverage} \times \text{Selection Reliability} \\ \times \text{Budget Efficiency}.\end{aligned}$$

任一因素接近零，额外推理计算的收益都会受限。

验证可靠性的开放问题

- 如何训练 outcome verifier 与 process verifier?
- 如何自动构造步骤级监督?
- 如何组合多个弱 verifier 来缩小选择差距?

参考文献

- Brown et al., *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*
- *How Do Large Language Monkeys Get Their Power (Laws)?*
- Snell et al., *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*
- Saad-Falcon et al., *Archon: An Architecture Search Framework for Inference-Time Techniques*